

Introduction to DevOps

A Foundational Guide

Chandu

August 2026

Contents

1. What is DevOps?	1
2. Why DevOps Matters	2
3. The DevOps Lifecycle	2
4. Key DevOps Practices	3
5. Common DevOps Toolchain	4
6. DevOps Culture: More Than Tools	4
7. A Simple CI/CD Pipeline Example	4
8. Getting Started: A Learning Path	5
9. Summary	5

1. What is DevOps?

DevOps is a set of practices, cultural philosophies, and tools that combines software **D**evelopment and IT **O**perations. Its goal is to shorten the software development life-cycle while delivering features, fixes, and updates frequently and reliably.

Before DevOps, development and operations teams worked in silos:

- **Developers** wrote code and threw it “over the wall” to operations.
- **Operations** teams deployed and maintained that code, often without understanding how it was built.
- This led to slow releases, finger-pointing during outages, and a disconnect between writing software and running it in production.

DevOps breaks down this wall by making one team (or closely collaborating teams) responsible for the entire lifecycle of an application — from writing code to deploying it, monitoring it, and fixing it in production.

Core Idea

“You build it, you run it.”

This principle, popularized by Amazon, means the people who write the code are also responsible for operating it in production. This creates strong incentives to write reliable, observable, and maintainable software.

2. Why DevOps Matters

Traditional Approach	DevOps Approach
Manual deployments, once every few months	Automated deployments, multiple times a day
Siloed Dev and Ops teams	Cross-functional collaboration
Manual testing before release	Automated testing in CI/CD pipelines
Reactive monitoring (find out from users)	Proactive monitoring and alerting
Infrastructure configured by hand	Infrastructure as Code (IaC)
Long, risky releases	Small, frequent, low-risk releases

Companies adopt DevOps to achieve:

1. **Faster time to market** — features reach users sooner.
 2. **Higher reliability** — automated testing and deployment reduce human error.
 3. **Better collaboration** — shared ownership between teams.
 4. **Faster recovery** — issues are detected and fixed quickly.
 5. **Scalability** — infrastructure can grow with demand through automation.
-

3. The DevOps Lifecycle

DevOps is often visualized as an infinite loop with eight phases:

1. **Plan** — Define requirements, features, and roadmap (tools: Jira, Trello).
2. **Code** — Write and review application code (tools: Git, GitHub, GitLab).
3. **Build** — Compile code and manage dependencies (tools: Maven, npm, Gradle).
4. **Test** — Run automated tests to catch bugs early (tools: JUnit, Selenium, Jest).
5. **Release** — Prepare a build for deployment (tools: Jenkins, GitHub Actions).
6. **Deploy** — Push the release to production (tools: Ansible, Kubernetes, Terraform).
7. **Operate** — Run and maintain the application in production (tools: AWS, Docker, Nginx).
8. **Monitor** — Track performance, logs, and errors (tools: Prometheus, Grafana, Loki).

This is a continuous loop — feedback from monitoring feeds back into planning, creating a cycle of constant improvement.

4. Key DevOps Practices

4.1 Continuous Integration (CI)

Developers frequently merge code changes into a shared repository. Each merge triggers an automated build and test process, catching integration issues early rather than at the end of a project.

4.2 Continuous Delivery / Deployment (CD)

- **Continuous Delivery:** Code changes are automatically prepared for a release but require manual approval before going to production.
- **Continuous Deployment:** Every change that passes automated tests is automatically deployed to production, with no manual step.

Together, CI/CD pipelines automate the path from a code commit to a live feature.

4.3 Infrastructure as Code (IaC)

Instead of manually configuring servers, infrastructure is defined in code (e.g., Terraform, CloudFormation). This makes environments reproducible, version-controlled, and easy to tear down and rebuild.

4.4 Configuration Management

Tools like Ansible, Chef, and Puppet automate the configuration of servers — installing packages, setting environment variables, and enforcing consistency across machines.

4.5 Containerization

Docker packages an application with everything it needs to run (code, runtime, libraries) into a single portable unit called a **container**. This eliminates the classic “it works on my machine” problem.

4.6 Orchestration

When you have many containers running across many servers, tools like **Kubernetes** manage their deployment, scaling, networking, and recovery automatically.

4.7 Monitoring and Observability

Once software is live, teams need visibility into how it behaves:

- **Metrics** — numeric data over time (e.g., Prometheus).
- **Logs** — detailed event records (e.g., Loki, ELK stack).

- **Dashboards** — visual representation of system health (e.g., Grafana).
 - **Alerting** — automatic notification when something goes wrong.
-

5. Common DevOps Toolchain

Category	Popular Tools
Version Control	Git, GitHub, GitLab, Bitbucket
CI/CD	Jenkins, GitHub Actions, GitLab CI, CircleCI
Configuration Management	Ansible, Chef, Puppet
Containerization	Docker, Podman
Orchestration	Kubernetes, Docker Swarm
Infrastructure as Code	Terraform, AWS CloudFormation
Cloud Platforms	AWS, Azure, Google Cloud Platform
Monitoring	Prometheus, Grafana, Datadog, Loki
Web/Reverse Proxy	Nginx, HAProxy

6. DevOps Culture: More Than Tools

DevOps is often misunderstood as “just a set of tools.” In reality, tools only support the underlying culture. Three cultural pillars matter most:

1. **Collaboration** — Dev, Ops, QA, and Security work together instead of in silos (this extended version is sometimes called **DevSecOps**).
2. **Automation** — Repetitive manual tasks (testing, deployment, provisioning) are automated to reduce human error and free up engineering time.
3. **Continuous Improvement** — Teams use metrics and feedback (deployment frequency, failure rate, recovery time) to constantly refine their processes.

The Blameless Postmortem

When something breaks in production, mature DevOps teams focus on **what** went wrong and **how** to prevent it — not **who** is to blame. This builds psychological safety and encourages honest reporting of issues.

7. A Simple CI/CD Pipeline Example

A typical pipeline for a web application might look like this:

1. A developer pushes code to a Git repository.

2. A CI tool (e.g., Jenkins or GitHub Actions) automatically triggers a build.
 3. Automated unit and integration tests run against the build.
 4. If tests pass, a Docker image is built and pushed to a container registry.
 5. The CD tool deploys the new image to a staging environment.
 6. After validation, the image is deployed to production — sometimes automatically, sometimes after manual approval.
 7. Monitoring tools track the new deployment's health and alert the team if errors spike.
-

8. Getting Started: A Learning Path

For someone starting out, a practical order to learn DevOps skills is:

1. **Linux fundamentals** — shell commands, file permissions, processes.
 2. **Git and version control** — branching, merging, pull requests.
 3. **Networking basics** — DNS, HTTP/HTTPS, load balancing.
 4. **A scripting language** — Bash or Python for automation.
 5. **Cloud fundamentals** — pick one provider (AWS, Azure, or GCP) and learn core services like compute, storage, and networking.
 6. **Containers** — Docker fundamentals, building and running images.
 7. **CI/CD** — set up a pipeline with Jenkins or GitHub Actions.
 8. **Configuration management and IaC** — Ansible and Terraform basics.
 9. **Orchestration** — Kubernetes fundamentals.
 10. **Monitoring** — Prometheus and Grafana basics.
-

9. Summary

DevOps is not a single tool or job title — it is a culture and set of practices that unify software development and operations to deliver software faster, more reliably, and with greater collaboration. It rests on three pillars: **automation**, **collaboration**, and **continuous feedback**. Mastering DevOps means gradually building competence across version control, automation, cloud infrastructure, containers, and monitoring — while internalizing a culture of shared ownership and continuous improvement.